Technical Report

End-to-End TinyML Deployment on Arduino UNO R4 WiFi (Renesas RA4M1)

A Complete, Reproducible Workflow with Explicit Model Contracts and Memory Budgeting: `.ipynb` → INT8 `.tflite` → Edge Impulse → Arduino

Amirreza Mehrabi

February 9, 2026

Contents

1 Executive Summary (What We Built, What We Proved, and Why It Matters)

This report presents a complete and reproducible workflow for deploying a small convolutional neural network (CNN) image classifier onto the Arduino UNO R4 WiFi, which is based on the Renesas RA4M1 microcontroller. The central engineering challenge in this project is not the conceptual training of a CNN model in Python, because modern machine learning frameworks make that stage relatively straightforward, but rather the practical execution of that trained model under the strict memory and compute constraints of a microcontroller. Specifically, the UNO R4 WiFi includes only 32 KB of SRAM, and this SRAM must simultaneously accommodate the input frame buffer, intermediate activation tensors produced during CNN inference, library state and global variables used by the runtime, and the stack space that is required for function calls during execution. For that reason, the deployment problem is fundamentally a memory budgeting problem, and successful deployment depends on ensuring that every byte of runtime memory consumption is planned and justified rather than assumed.

In this workflow, we intentionally use the Edge Impulse deployment pipeline because it provides two essential capabilities that reduce risk on memory-limited boards. First, it forces us to explicitly define and confirm the model input and output contract, including the input tensor shape, the scaling rule applied to pixels, the resize strategy, the output interpretation (classification), and the label ordering, which prevents a common class of errors where the firmware runs but the predictions are meaningless. Second, it exports an Arduino library that packages a microcontroller-oriented inference runtime with a stable interface, including the `signal_t` abstraction and the `run_classifier` function, which allows us to focus on correct data feeding and memory-safe buffering rather than on assembling a complex TensorFlow Lite Micro build configuration manually. The board specifications and the recommended board installation methods are grounded in Arduino's official documentation, and the Arduino library export path is aligned with Edge Impulse's published instructions (Arduino, n.d., 2024; Edge Impulse, n.d.).

2 System Architecture and Definitions (A Clear Model of What Depends on What)

2.1 End-to-End Pipeline Overview

The full pipeline is best understood as a sequence of stages that each produce a specific artifact and each introduce a distinct class of failure modes. The purpose of presenting the pipeline explicitly is to avoid "mixed debugging," where a memory problem on the board is mistakenly treated as a training issue, or an input-scaling issue is mistakenly treated as a code compilation issue. Figure 1 provides the map of the workflow that this report follows, and it should be read as the top-level reference diagram for the remainder of the document.

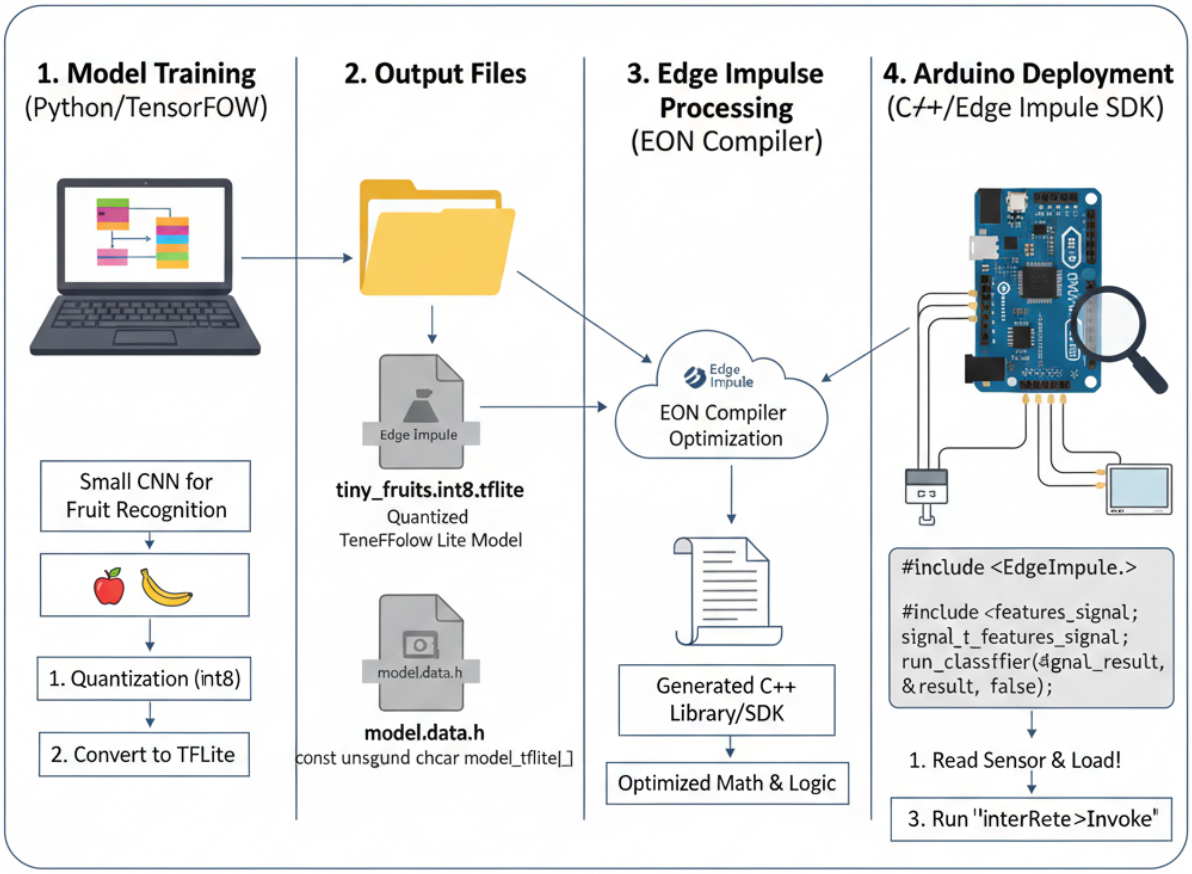

Figure 1: End-to-end workflow for deploying an INT8-quantized CNN on the Arduino UNO R4 WiFi using Edge Impulse. The pipeline separates (1) model training and quantization, (2) production of deployable artifacts (`.tflite` and an optional model header), (3) Edge Impulse processing and code generation (EON Compiler), and (4) Arduino-side integration and inference execution.

Figure 1 is not only a visual summary; it is also a statement of responsibilities. In this report, the Python notebook is responsible for producing a numerically correct and microcontroller-feasible `.tflite` artifact, while Edge Impulse is responsible for locking the model contract and generating a stable inference library interface, and the Arduino sketch is responsible for feeding input in the correct format and running inference without violating the SRAM budget. This stage separation is necessary because each stage has different correctness criteria, and confusing those criteria is one of the main reasons TinyML projects fail in practice.

2.2 Two Complementary Layers: Modeling and Embedded Implementation

In this project, the deployment pipeline is most accurately described as a two-layer architecture in which the modeling layer and the embedded implementation layer solve fundamentally different problems while still depending on each other. The modeling layer is responsible for producing a correct machine learning model and exporting it into a portable artifact that can be executed in a non-Python environment, while the embedded implementation layer is responsible for integrating that artifact into firmware that can execute reliably within the board's memory and compute limits. This separation is important because it prevents the common mistake of treating embedded inference as if it were simply "running the Python model on a smaller computer," when in reality embedded inference is a constrained systems problem that requires strict control over buffers, intermediate memory, and runtime initialization.

2.3 What the "Model Contract" Means and Why It Is Non-Negotiable

When we say that the model has a "contract," we mean that the model defines an input interface and an output interface that must be respected exactly in the embedded firmware. The input interface includes the tensor shape, channel semantics, datatype expectations, and scaling rule, and the output interface includes the output vector size, the interpretation of that output (classification vs. regression), and the label order that maps output indices to human-readable class names. If any part of this contract is violated by the Arduino firmware, the system may still compile successfully and may still run without crashing, but the inference results cannot be interpreted as valid measurements of the intended classification function because the model is being evaluated on data that is not in the distribution or format that it was trained to consume.

3 Model Architecture (What the CNN Computes and Why This Design is Microcontroller Feasible)

3.1 High-Level Architecture Summary

The small CNN in this project is designed to be computationally light while still extracting useful features from low-resolution images. Figure 2 presents the architecture at a level that is sufficient for both ML understanding (what layers exist and what they do) and embedded feasibility analysis (why the network is not too large to run on a microcontroller). The most important architectural idea in this model is the use of separable convolutions, which typically reduce parameter count and compute compared to standard convolutions, and this reduction is particularly valuable when inference must execute on a 48 MHz Cortex-M4F.

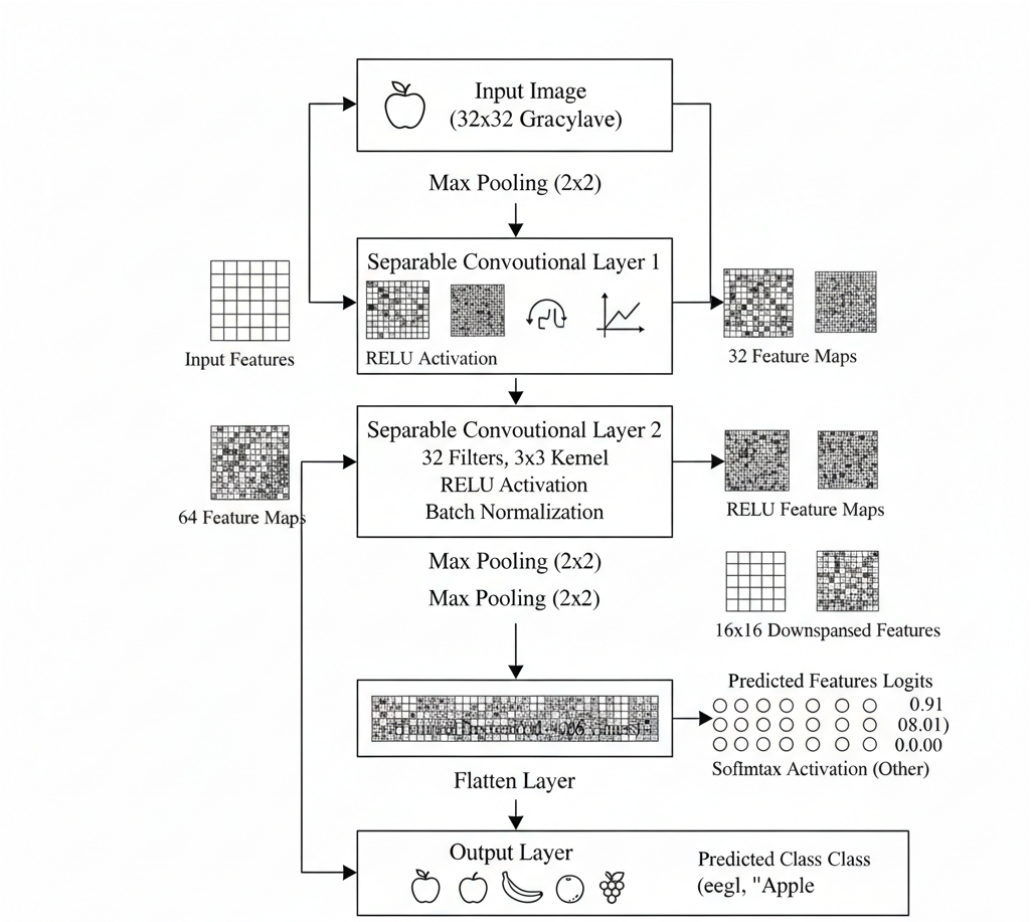

Figure 2: Small CNN architecture used for fruit recognition. The design uses separable convolution blocks to reduce parameter count and multiply-accumulate operations relative to conventional convolutions, followed by a compact dense classifier head and a softmax output over classes.

Figure 2 should be interpreted in two complementary ways. From a modeling perspective, the early convolution layers act as feature extractors that transform the raw image into a set of learned feature maps, and the later dense layer maps these extracted features into class logits. From a deployment perspective, each convolution stage also creates intermediate activation tensors, and these intermediate tensors are the largest runtime memory consumers during inference. Therefore, even if the weight storage is small after INT8 quantization, the architecture still matters because it determines the peak activation footprint that must fit inside SRAM.

3.2 Feature Extraction Intuition (What Feature Maps Represent, and Why We Mention Them)

A frequent gap in embedded ML explanations is that they describe "the model runs" without explaining what the intermediate layers are doing, which makes it harder to justify why certain memory allocations are unavoidable. Figure 2 provides an intuition-level visualization of feature extraction, which helps connect CNN design to both accuracy (features become progressively more discriminative) and memory budgeting (feature maps are transient tensors that occupy SRAM during inference).

Figure 2 is intentionally conceptual, meaning that it visualizes the feature extraction process rather than providing a strict, line-by-line equivalence to the implementation graph exported in the `.tflite` file. The essential technical point is that convolution layers produce multiple feature maps that represent different learned responses to local patterns, and these feature maps are allocated as intermediate activations inside the runtime tensor arena. Because the tensor arena must accommodate the maximum set of simultaneously needed activations, models that grow feature maps too aggressively or preserve high spatial resolution too long can exceed SRAM even if the total number of parameters is modest. For that reason, pooling and compact layer choices are not only about generalization; they are also about limiting peak activation memory.

4 What the `.ipynb` Notebook Does (Precise Technical Purpose, Explained Fully)

4.1 The Notebook's Primary Job: Converting a Trainable Model into a Deployable Artifact

The notebook (`.ipynb`) is the formal model production pipeline, and its purpose is to convert a model that is initially trainable and testable in a high-level Python environment into a model artifact that can be executed on a microcontroller using a C/C++ inference runtime. In practice, that means the notebook must output a deployable representation that preserves both the numerical parameters of the model (weights and biases) and the computation graph (layer sequence, tensor shapes, and operator definitions). In this workflow, the deployable representation is the INT8 quantized TensorFlow Lite file saved as `tiny_fruits_int8.tflite`, and this file is treated as the canonical model artifact that is uploaded into Edge Impulse.

4.2 Dataset Ingestion and the Importance of Stable Label Indexing

A correct notebook first loads the dataset and ensures that each sample has a consistent label that is mapped into a stable integer index. This step is essential because the final classifier produces an output vector where each element corresponds to a specific class index, and any mismatch between training-time label indexing and deployment-time label ordering results in outputs that look numerically reasonable but correspond to the wrong classes. Therefore, a robust notebook explicitly defines a deterministic class list, enforces consistent label-to-index mapping, and ensures that the output layer dimension matches the number of classes exactly.

4.3 Input Tensor Definition and Why the Shape Cannot Be Treated as a Minor Detail

The notebook defines the model input tensor shape and trains the model to accept that exact shape, which means the deployed model cannot be safely executed with a different input resolution or channel configuration without retraining or re-exporting. In your deployment configuration, the input tensor shape is `(32, 32, 1)`, which indicates a 32-by-32 image with a single channel, and this typically corresponds to a grayscale representation where each pixel contributes exactly one numeric value. This design choice directly determines the microcontroller input buffer size, and it also influences the sizes of intermediate activations because early convolutional layers expand channel depth based on the input and the first layer's filter configuration.

4.4 Preprocessing Contract: Scaling Pixels to 0..1 and Why It Must Match at Inference Time

The notebook either explicitly rescales pixels during training or embeds an assumption about the scaling that will be applied before inference. Your configuration specifies that the input should be scaled as "Pixels ranging 0..1 (not normalized)," which means the firmware must provide values between 0 and 1, typically by computing $x = \text{pixel}/255$. This scaling is a structural requirement because CNN filters are learned relative to the numerical distribution of their inputs, and changing the scale changes activations throughout the network. As a result, if firmware feeds 0..255 values into a model trained on 0..1, the model may saturate and collapse to a near-constant output distribution, and if firmware feeds -1..1 into a model trained on 0..1, the model effectively receives inputs from a different distribution and becomes unreliable.

4.5 Why INT8 Quantization Is a Microcontroller Necessity Rather Than an Optional Optimization

INT8 quantization is used because microcontrollers have limited Flash and limited SRAM, and quantization substantially reduces the storage required for weights while also enabling efficient integer kernels in embedded runtimes. From an embedded systems perspective, INT8 quantization helps in two ways: it reduces the model footprint stored in Flash and it can reduce runtime memory pressure when the inference engine allocates quantized activation buffers rather than float buffers. In this workflow, `tiny_fruits_int8.tflite` is treated as the correct deployable artifact because it aligns with the UNO R4's strict memory limits more reliably than a float model.

4.6 Optional `.h` Export and Why We Embed Model Bytes in Firmware

On Arduino platforms, a common pattern is to convert the `.tflite` file into a C/C++ array and compile it into the firmware, which stores model bytes in Flash and avoids any need for external storage. When the notebook outputs a header (for example, `model_data.h`), that header typically contains a `const unsigned char` array and its length, and the firmware includes that header so the model bytes are available at compile time. This approach is appropriate because Flash is intended to hold static program data, whereas SRAM must remain available for working memory, including the tensor arena and stack.

5 The Target Board: UNO R4 WiFi (RA4M1) and the Exact Reason SRAM Dominates

5.1 UNO R4 WiFi as a Microcontroller Deployment Target

The Arduino UNO R4 WiFi is best understood as a constrained microcontroller platform rather than a small general-purpose computer, because it has fixed SRAM and Flash resources and does not provide an operating system with large virtual memory. The practical consequence of this architecture is that inference feasibility is determined not only by whether the model fits in Flash, but also by whether the runtime activations and stack requirements fit in SRAM without collision. Arduino's official documentation provides the baseline board description and specifications used to justify the constraints in this report (Arduino, n.d.).

5.2 Board Feature Table (TinyML-Relevant)

Table 1: Arduino UNO R4 WiFi (Renesas RA4M1) — Features That Matter for TinyML

Feature	Why It Matters for This Deployment
Cortex-M4F @ 48 MHz	This MCU class and frequency bound inference latency and throughput on-device.
SRAM: 32 KB	SRAM must hold input buffers, activation tensors, runtime state, and stack simultaneously, so
Flash: 256 KB	Flash stores the firmware and embedded model weights, so it bounds code + model size feasib
Floating Point Unit (FPU)	The FPU can accelerate some float math, but it does not remove SRAM constraints or tensor a
WiFi capability	WiFi can support optional logging or communication, but it is not required for baseline offline

5.3 How Memory is Actually Used During Inference (Flash vs SRAM, and Why "Tensor Arena" is Central)

When we discuss memory on UNO R4 WiFi, we must distinguish between Flash and SRAM because they support different roles during inference. Flash stores the compiled program and static data such as embedded model weights, while SRAM stores all working data created during execution. This distinction matters because almost all instability in microcontroller inference is caused by SRAM exhaustion, not Flash exhaustion, since SRAM must store the input tensor, intermediate activation tensors, runtime state, and the stack.

Figure 3 provides a concrete memory map and inference-cycle diagram that connects memory regions to the steps of execution. This figure is inserted here because the purpose of the next paragraphs is to explain, precisely and concretely, why RAM is the true limiting factor in practice.

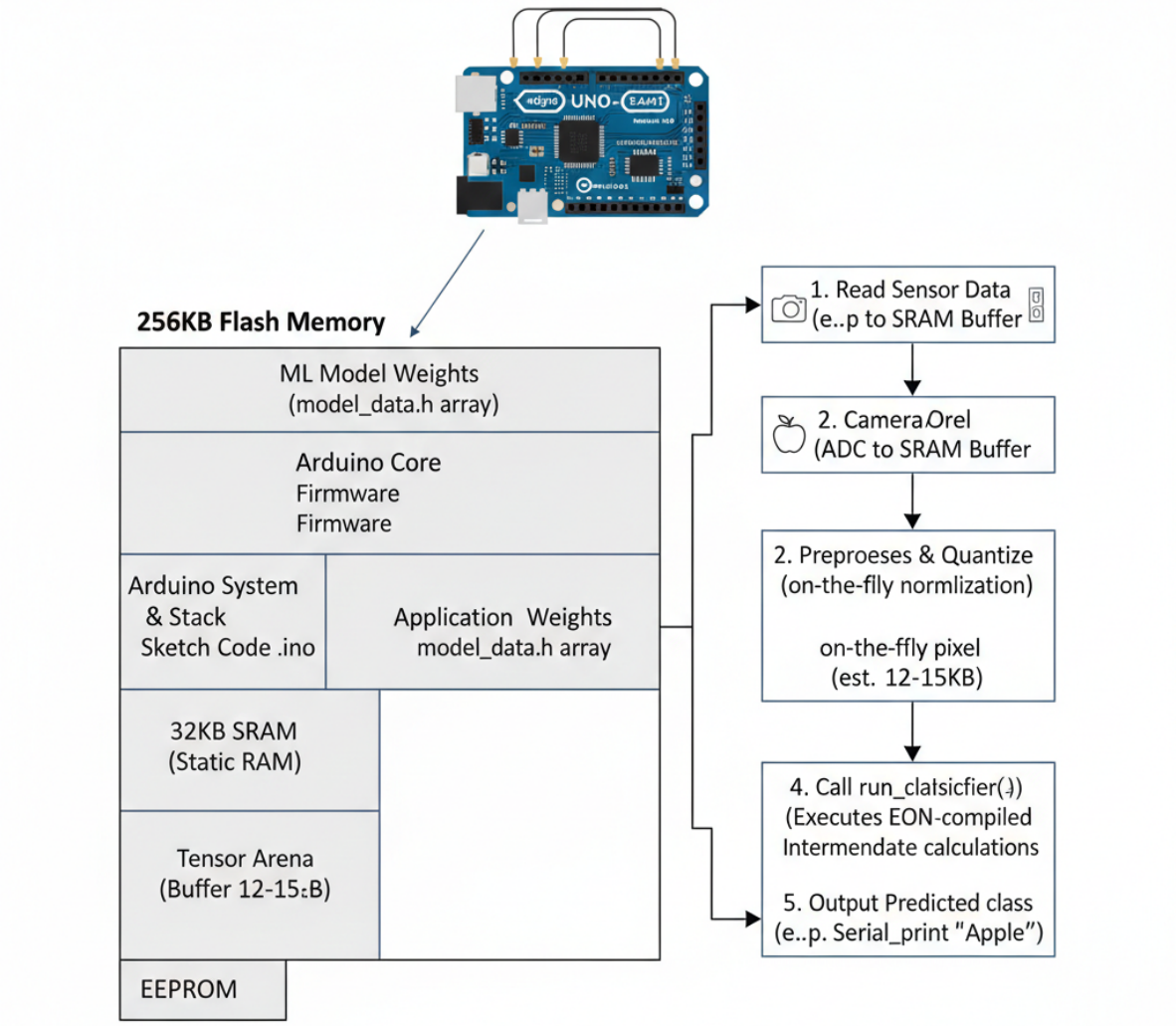

Figure 3: UNO R4 WiFi memory map and TinyML inference cycle. Model weights and firmware reside in Flash, while the input buffer, tensor arena (activation scratch space), runtime globals, and the call stack must fit within the 32 KB SRAM budget.

Figure 3 highlights the most important embedded ML concept for this project, which is that the tensor arena is the runtime scratch space that holds intermediate activations and temporary tensors used by the inference engine. Even for small CNNs, convolution layers can generate feature maps whose combined memory footprint is multiple times larger than the input buffer, especially before pooling reduces spatial resolution. Because the tensor arena must support the maximum set of simultaneously live tensors, a model can fail on SRAM even when its weights easily fit in Flash. In addition, the stack must remain available for function calls and library code paths, which means it is unsafe to allocate nearly all SRAM to global buffers or to an oversized arena, because that leaves insufficient headroom and can cause unpredictable resets.

5.4 Concrete Input Memory and Why `uint8_t` Buffers Are Safer

For an input tensor of shape (H, W, C), the buffer size depends on datatype, because a `uint8` value requires 1 byte while a `float32` value requires 4 bytes. For `(32, 32, 1)`, the raw input is 1024 bytes as `uint8_t` but becomes 4096 bytes as `float32`. Although 4096 bytes is not large by desktop standards, it is significant on a 32 KB microcontroller once the tensor arena and stack are included, which is why memory-safe designs store the raw input as `uint8_t` and perform scaling only when the inference engine requests input data.

6 Why We Use Edge Impulse and Why We Do Not Install Generic "Original" Libraries

6.1 Edge Impulse as a Toolchain That Reduces Integration Risk

We use Edge Impulse because it reduces two dominant failure modes in microcontroller inference projects: contract mismatch and runtime mismatch. Contract mismatch occurs when the model is trained with one preprocessing pipeline but firmware provides different scaling, channel order, or shape. Runtime mismatch occurs when a generic inference runtime is assembled incorrectly for a given Arduino core, leading to missing headers, incompatible build flags, unsupported operator sets, or memory defaults that are not feasible under 32 KB SRAM. By uploading the `.tflite`, explicitly confirming the contract, and exporting an Arduino library that compiles inside Arduino's build system, Edge Impulse reduces the size and complexity of the integration surface (Edge Impulse, n.d.).

6.2 Why Generic TensorFlow Lite Arduino Installs Commonly Fail on UNO-Class Boards

Generic TensorFlow Lite Arduino installs frequently fail on UNO-class boards because they tend to assume a development environment where one can include a broad operator set and allocate large working buffers. Even when compilation succeeds, default configurations can allocate too much SRAM, leaving insufficient headroom for stack growth, which can cause runtime instability. Edge Impulse's Arduino export avoids many of these issues because it packages the runtime and exposes only the required entry points (`run_classifier` and related structures) in a way that is designed to work with Arduino's library and build conventions (Edge Impulse, n.d.).

7 Step-by-Step Procedure (Exact Click-Path Workflow, With Technical Reasons)

This section is written as an operational checklist that you can follow on a real machine. Each step names the software, the expected files produced, the most common failure mode at that step, and why the step exists from a TinyML systems perspective.

7.1 Step 1: Download and Install the Arduino IDE (Toolchain Baseline)

You must first install the Arduino IDE because the UNO R4 WiFi is compiled and flashed through Arduino's board platform toolchain, not through a generic CMake toolchain. In practice, Arduino IDE 2.x is recommended because it supports modern Boards Manager workflows and reliably registers imported ZIP libraries. At the end of this step, you should be able to open Arduino IDE and see `Tools > Board` and `Tools > Port` menus, even if UNO R4 is not installed yet.

7.2 Step 2: Create a Single Working Folder and Keep All Artifacts Together (Path Stability)

You must create one working directory that contains your notebook (`.ipynb`), exported artifacts (`.tflite` and `.h`), and your Arduino sketch (`.ino`). This matters because Arduino resolves header includes relative to the sketch folder, and scattered files are the most common reason for "file not found" and silent version confusion. Concretely, you should have one folder where, at minimum, you can see the following files at the end of the workflow:

- `your_notebook.ipynb`
- `tiny_fruits_int8.tflite`
- `ipynbOutputthatHavedotH.h` (your C-array header that stores model bytes)
- `your_arduino_sketch.ino`

7.3 Step 3: Install the One Deployment Library We Actually Use (Why This Specific Library)

You should not start by installing generic TensorFlow Lite Arduino libraries, because they often fail on UNO-class SRAM budgets or fail due to board-core incompatibilities, missing headers, or unsupported operator builds. Instead, you will use the Edge Impulse Arduino export, which is designed as a microcontroller-friendly SDK and is meant to be imported as a ZIP library. This step is therefore a planning step: the required library is not installed from the Arduino Library Manager first; it is installed later (Step 10–11) from Edge Impulse as a ZIP so the engine and header names match exactly.

7.4 Step 4: Run the `.ipynb` End-to-End (Model Export Pipeline)

You must run the notebook from top to bottom because the notebook is not "just training"; it is the production pipeline that exports a deployable model artifact. In technical terms, the notebook is responsible for generating a TensorFlow Lite model file whose operators and numeric types are compatible with microcontroller inference, and it is also responsible for quantization decisions that determine whether deployment fits into Flash and SRAM. If you stop the notebook early, you often end up with either (a) no `.tflite` file, or (b) a float `.tflite` file that is significantly harder to run on UNO-class memory budgets.

7.5 Step 5: Confirm You Have the Correct `.tflite` Output (The Artifact We Upload)

After the notebook finishes, you must locate the exported INT8 file and verify its name and location. In your workflow, the intended deployment artifact is:

`tiny_fruits_int8.tflite`

This specific file name matters because it is the file you upload to Edge Impulse in Step 6–7, and it is also the file referenced in the Edge Impulse "Process model" step text. If you upload a different `.tflite` (such as a float version), you may still be able to build a library, but SRAM usage and runtime feasibility can change drastically.

7.6 Step 6: Create an Edge Impulse Account and Upload the `.tflite` (Why the Website Is Needed)

You must use Edge Impulse because it does two things that reduce failure risk on UNO R4 WiFi. First, it forces you to explicitly confirm the model contract (input shape, scaling, output labels), which prevents silent preprocessing mismatches that produce meaningless predictions. Second, it builds an Arduino library that packages a compatible inference runtime and a stable API (`run_classifier`, `signal_t`) without requiring you to manually assemble TensorFlow Lite Micro configurations. You therefore upload `tiny_fruits_int8.tflite` into Edge Impulse so it can generate the exact Arduino-ready deployment package (Edge Impulse, n.d.).

7.7 Step 7: Confirm the Model Contract in Edge Impulse Deployment (Verbatim Settings)

Step 2: Process "tiny_fruits_int8.tflite"
Configure model settings for optimal processing.
Model input
Input shape: (32, 32, 1)

Image
How is your input scaled?

Pixels ranging 0..1 (not normalized)
Input should be in RGB format (one value per pixel). If your model uses a different channel order, or is scaled differently, then select "Other".
Resize mode

Squash
Input data will be automatically resized to match the model input shape using this mode. For best accuracy, upload pre-processed testing data.
Model output
Output shape: (6)

Classification
Output labels (6)
Enter labels for your model separated by ','.
class 1, class 2, class 3, class 4, class 5, class 6

Explanation (Contract Enforcement): You must treat this configuration as the authoritative specification of the model interface because it defines the exact numerical tensor that firmware must provide and the exact semantic mapping for outputs. If any of these settings change, then the exported Arduino library should be rebuilt and treated as a new deployment artifact, because preprocessing and wrapper assumptions may differ.

Important Input-Format Consistency Note (Grayscale vs RGB): The input shape `(32, 32, 1)` indicates a single-channel tensor, which corresponds to grayscale semantics where each pixel contributes one value. The Edge Impulse interface sometimes displays generic text about "RGB format," but for a 1-channel model the firmware must not silently expand the input to three channels unless the model was trained and exported with `(32, 32, 3)`. If channel expansion occurs without retraining, the numeric distribution and tensor semantics change, and classification performance can degrade even when inference runs without errors.

7.8 Step 8: Install the UNO R4 Platform via Boards Manager URL (So Arduino Can Compile for RA4M1)

You must add the UNO R4 boards package URL in Arduino IDE because the UNO R4 WiFi requires a specific core, compiler configuration, and board definition files that Arduino downloads through Boards Manager. Concretely, you open:

`File > Preferences > Additional Boards Manager URLs`

and you add the following URL exactly:

`https://downloads.arduino.cc/packages/package_UNOr4_earlyaccess_index.json`

You then open:

`Tools > Board > Boards Manager`

search for UNO R4, and install the UNO R4 platform. This step matters because without the correct board platform, Arduino cannot compile and link code for the RA4M1 toolchain (Arduino, 2024).

7.9 Step 9: Put the Notebook-Exported Model Header (`.h`) Into the Arduino Sketch Folder (So the Model Lives in Flash)

You must copy the notebook output header file (the one that contains a C array of model bytes) into the exact same folder as your `.ino` sketch file. This step matters because Arduino compiles everything in the sketch folder together, and it resolves local headers (quoted includes) from that folder. Technically, this header is how the model weights are embedded into Flash as a static byte array, which prevents wasting SRAM on model storage.

7.10 Step 10: Download the Edge Impulse Arduino Library as a ZIP (This Is the Inference Engine)

In Edge Impulse Studio, you go to the **Deployment** tab, select **Arduino Library**, and click **Build**. This produces a ZIP file download such as:

`ei-modeh-arduino-1.0.1.zip`

You must record the ZIP filename (or the library name inside it) because the correct include header you will use in the `.ino` depends on the library's installed header name. This ZIP contains the inference engine and its required types and functions, including `signal_t` and `run_classifier` (Edge Impulse, n.d.).

7.11 Step 11: Import the ZIP Library, Create the `.ino`, and Include Both Headers Correctly

You must import the Edge Impulse ZIP into Arduino IDE by selecting:

`Sketch > Include Library > Add .ZIP Library...`

After import, the library becomes available for compilation, and Arduino can resolve its headers through angle-bracket includes. Next, you create your Arduino sketch (`.ino`) in the *same folder* that contains your notebook-exported `.h` file (Step 9), and at the very top of the sketch you include exactly two headers: (1) the Edge Impulse engine header from the ZIP library, and (2) your notebook-exported model header. The include names must match the real files exactly; otherwise, `signal_t` and `run_classifier` will not compile.

```
#include <Arduino.h>

/**
 * 1. FIX: You MUST change this name to match the EXACT .h file
 * provided by the imported Edge Impulse ZIP library.
```

```
 * If this name is wrong, 'signal_t' and 'run_classifier' will fail.
 */
#include <NameOfZIPFile.h>            // The Edge Impulse engine (from ZIP
    library)
#include "ipynbOutputthatHavedotH.h"  // The notebook-exported model byte array
    (local file)
```

Listing 1: Required includes at the top of the Arduino sketch (engine + model bytes)

After saving the `.ino`, you compile and upload to the UNO R4 WiFi, and you open the Serial monitor to confirm the program runs without resets. If inference runs but outputs do not change, you should treat that outcome as an input contract or data-feeding problem rather than as a failure of the model itself.

8 Troubleshooting With Complete Technical Causes and Correct Fixes

8.1 If `signal_t` or `run_classifier` Is Not Found

If the compiler reports that `signal_t` or `run_classifier` is not declared, then the most likely explanation is that the header included by `#include <NameOfZIPFile.h>` does not match the header provided by the imported Edge Impulse ZIP library. You should locate the exact header name inside the installed library and update the include directive to match it precisely, because Arduino include resolution depends on library names and header filenames.

8.2 If the Board Runs but Predictions Are Constant or Incorrect

If the board executes inference but always reports the same class or produces a non-changing output distribution, then the most likely explanation is that the input tensor does not match the contract defined in Step 7. In practice, this usually occurs because scaling is incorrect (0..255 instead of 0..1), because the buffer is not populated with real data (e.g., it contains zeros), or because grayscale vs RGB semantics were incorrectly handled. You should validate that you are producing exactly `(32, 32, 1)` values and that scaling produces floats in 0..1 during inference input access.

8.3 If the Board Resets or Behaves Unstably

If the board resets during inference or behaves inconsistently, then the most likely explanation is SRAM exhaustion caused by oversized buffers, an oversized tensor arena, or insufficient stack headroom. You should reduce SRAM usage by eliminating unnecessary globals, avoiding float copies of the input buffer, and ensuring the model bytes are stored in Flash and defined only once. You should interpret Figure 3 as the guiding model for what must fit into SRAM at runtime and why leaving headroom is necessary for stability.

9 References (APA)

References

Arduino. (n.d.). *UNO R4 WiFi: Documentation*. Arduino Docs. https://docs.arduino.cc/hardware/uno-r4-wifi

Arduino. (2024). *Add third-party platforms to the Boards Manager in Arduino IDE*. Arduino Support. https://support.arduino.cc/hc/en-us/articles/360016466340-Add-third-party-platforms-to-the-Boards-Manager-in-Arduino-IDE

Edge Impulse. (n.d.). *Run Arduino library (IDE 2.0)*. Edge Impulse Documentation. https://docs.edgeimpulse.com/hardware/deployments/run-arduino-2-0